

GYKHAMINE CAPSULE – ROADMAP COMPLÈTE
DOMAINE : ÉCOLE / ÉTABLISSEMENT SCOLAIRE
Base de départ : Boutique ERP → École Manager
Contexte : École primaire ou secondaire, Congo, usage hors-ligne

PHASE 0 – ANALYSE DU DOMAINE (Jour 1, sur papier)

ENTITÉS DU DOMAINE ÉCOLE :

- Élève (inscrit dans l'école)
- Classe (6ème A, Terminale D, CP2...)
- Enseignant / Professeur
- Matière (Mathématiques, Français, SVT...)
- Emploi du temps (cours programmés)
- Note / Évaluation (devoir, composition, examen)
- Bulletin de notes (relevé trimestriel)
- Présence / Absence (suivi assiduité)
- Paiement des frais scolaires (scolarité, tenue, manuels)
- Parent / Tuteur légal
- Discipline (incidents, sanctions)
- Réunion de parents d'élèves

CORRESPONDANCE BOUTIQUE ERP → ÉCOLE MANAGER :

BOUTIQUE ERP (original)	ÉCOLE MANAGER (nouveau domaine)
Produit	Élève
Categorie	Classe (6ème A, 5ème B...)
Client	Parent / Tuteur légal
Fournisseur	Enseignant / Professeur
Vente	Paiement de frais scolaires
LigneDeVente	Ligne de frais (scolarité, tenue...)
Bon de commande	Bulletin de notes trimestriel
Stock	Capacité d'accueil par classe
Rapport mensuel ventes	Rapport trimestriel résultats scolaires
Dashboard chiffre d'aff	Dashboard vie scolaire

PHASE 1 – PRÉPARATION DU PROJET (Jour 1-2)

ÉTAPE 1.1 – Dupliquer la capsule

```
cp -r Boutique_erp/ Ecole_Manager/  
cd Ecole_Manager/
```

ÉTAPE 1.2 – Renommer le dossier applicatif

```
mv boutique/ ecole/
```

ÉTAPE 1.3 – Adapter settings.py

```
INSTALLED_APPS : remplacer 'boutique' par 'ecole'  
LANGUAGE_CODE  = 'fr-FR'  
TIME_ZONE      = 'Africa/Brazzaville'  
DEBUG          = False
```

```
# Ajouter en bas de settings.py :
```

```
NOM_ECOLE      = "Groupe Scolaire Saint-Exupéry"  
DEVISE_ECOLE   = "Savoir, Rigueur, Excellence"  
ANNEE_SCOLAIRE = "2024-2025"  
MONNAIE        = "FCFA"  
FRAIS_SCOLARITE = 45000 # Montant annuel en FCFA
```

ÉTAPE 1.4 – Adapter urls.py racine

```
Remplacer : include('boutique.urls')  
Par       : include('ecole.urls')
```

ÉTAPE 1.5 – Supprimer migrations et base existante

```
rm ecole/migrations/0*.py  
rm db.sqlite3
```

PHASE 2 – RÉÉCRITURE DE models.py (Jour 2-3)

```

from django.db import models
from django.core.validators import MinValueValidator, MaxValueValidator

class Classe(models.Model):
    NIVEAUX = [
        ('cp1', 'CP1'), ('cp2', 'CP2'),
        ('ce1', 'CE1'), ('ce2', 'CE2'),
        ('cm1', 'CM1'), ('cm2', 'CM2'),
        ('6e', '6ème'), ('5e', '5ème'), ('4e', '4ème'), ('3e', '3ème'),
        ('2de', '2nde'), ('1ere', '1ère'), ('tle', 'Terminale'),
    ]
    SERIES = [
        ('', 'Sans série'), ('a', 'Série A'), ('b', 'Série B'),
        ('c', 'Série C'), ('d', 'Série D'),
    ]
    niveau = models.CharField(max_length=5, choices=NIVEAUX)
    serie = models.CharField(max_length=2, choices=SERIES, blank=True)
    lettre = models.CharField(max_length=2, blank=True,
                             verbose_name="Lettre (A, B, C...)")
    effectif_max = models.IntegerField(default=50)
    salle = models.CharField(max_length=20, blank=True)

    def __str__(self):
        label = self.get_niveau_display()
        if self.serie: label += f" {self.get_serie_display()}"
        if self.lettre: label += f" {self.lettre}"
        return label
    class Meta:
        verbose_name = "Classe"
        ordering = ['niveau', 'lettre']

class Tuteur(models.Model):
    nom = models.CharField(max_length=100)
    prenom = models.CharField(max_length=100)
    telephone = models.CharField(max_length=20)
    telephone2 = models.CharField(max_length=20, blank=True)
    adresse = models.TextField(blank=True)
    profession = models.CharField(max_length=100, blank=True)

    def __str__(self): return f"{self.nom} {self.prenom}"
    def nom_complet(self): return f"{self.prenom} {self.nom}"
    class Meta: verbose_name = "Parent / Tuteur"

class Eleve(models.Model):
    SEXES = [('M', 'Masculin'), ('F', 'Féminin')]
    STATUTS = [
        ('inscrit', 'Inscrit'),
        ('redoublant', 'Redoublant'),
        ('transfere', 'Transféré'),
        ('exclu', 'Exclu'),
        ('diplome', 'Diplômé / Sorti'),
    ]
    matricule = models.CharField(max_length=20, unique=True)
    nom = models.CharField(max_length=100)
    prenom = models.CharField(max_length=100)
    sexe = models.CharField(max_length=1, choices=SEXES)
    date_naissance = models.DateField()
    lieu_naissance = models.CharField(max_length=100, blank=True)
    classe = models.ForeignKey(Classe, on_delete=models.PROTECT)
    tuteur = models.ForeignKey(Tuteur, on_delete=models.SET_NULL,
                              null=True, blank=True)
    date_inscription = models.DateField()
    statut = models.CharField(max_length=15, choices=STATUTS,
                              default='inscrit')
    photo = models.ImageField(upload_to='eleves/', null=True, blank=True)
    boursier = models.BooleanField(default=False)
    notes_interne = models.TextField(blank=True)

    def __str__(self): return f"{self.nom} {self.prenom} ({self.classe})"
    def nom_complet(self): return f"{self.prenom} {self.nom}"

```

```

def age(self):
    from datetime import date
    today = date.today()
    return today.year - self.date_naissance.year - (
        (today.month, today.day) < (self.date_naissance.month, self.date_naissance.day))

class Meta:
    verbose_name = "Élève"
    ordering = ['classe', 'nom', 'prenom']

class Enseignant(models.Model):
    nom = models.CharField(max_length=100)
    prenom = models.CharField(max_length=100)
    telephone = models.CharField(max_length=20, blank=True)
    email = models.EmailField(blank=True)
    specialite = models.CharField(max_length=100, blank=True)
    actif = models.BooleanField(default=True)

    def __str__(self): return f"{self.prenom} {self.nom}"
    class Meta: verbose_name = "Enseignant"

class Matiere(models.Model):
    nom = models.CharField(max_length=100)
    code = models.CharField(max_length=10, blank=True)
    coefficient = models.DecimalField(max_digits=3, decimal_places=1, default=1.0)
    enseignants = models.ManyToManyField(Enseignant, blank=True)

    def __str__(self): return self.nom
    class Meta: verbose_name = "Matière"

class Note(models.Model):
    TRIMESTRES = [('1', '1er trimestre'), ('2', '2ème trimestre'), ('3', '3ème trimestre')]
    TYPES_EVAL = [
        ('devoir', 'Devoir surveillé'),
        ('composition', 'Composition'),
        ('interrogation', 'Interrogation orale'),
        ('tp', 'Travaux pratiques'),
        ('examen', 'Examen'),
    ]
    eleve = models.ForeignKey(Eleve, on_delete=models.CASCADE,
                              related_name='notes')
    matiere = models.ForeignKey(Matiere, on_delete=models.PROTECT)
    trimestre = models.CharField(max_length=1, choices=TRIMESTRES)
    type_eval = models.CharField(max_length=15, choices=TYPES_EVAL,
                                 default='devoir')
    note = models.DecimalField(max_digits=4, decimal_places=2,
                               validators=[MinValueValidator(0), MaxValueValidator(20)])
    date_eval = models.DateField()
    commentaire = models.CharField(max_length=200, blank=True)

    def __str__(self):
        return f"{self.eleve} – {self.matiere} : {self.note}/20"
    class Meta:
        verbose_name = "Note"
        ordering = ['-date_eval']

class AbsenceEleve(models.Model):
    eleve = models.ForeignKey(Eleve, on_delete=models.CASCADE,
                              related_name='absences')
    date = models.DateField()
    heure_debut = models.TimeField(null=True, blank=True)
    heure_fin = models.TimeField(null=True, blank=True)
    justifiee = models.BooleanField(default=False)
    motif = models.CharField(max_length=200, blank=True)

    def __str__(self):
        j = "justifiée" if self.justifiee else "non justifiée"
        return f"{self.eleve} – {self.date} ({j})"
    class Meta:
        verbose_name = "Absence"
        ordering = ['-date']

class TypeFrais(models.Model):
    nom = models.CharField(max_length=100)
    montant_std = models.DecimalField(max_digits=10, decimal_places=0)
    obligatoire = models.BooleanField(default=True)

```

```

def __str__(self): return f"{self.nom} ({self.montant_std} FCFA)"
class Meta: verbose_name = "Type de frais"

class Paiement(models.Model):
    MODES = [('especes', 'Espèces'), ('mobile', 'Mobile Money'), ('cheque', 'Chèque')]
    eleve = models.ForeignKey(Eleve, on_delete=models.PROTECT)
    type_frais = models.ForeignKey(TypeFrais, on_delete=models.PROTECT)
    montant = models.DecimalField(max_digits=10, decimal_places=0)
    date = models.DateField()
    mode = models.CharField(max_length=10, choices=MODES, default='especes')
    recu_numero = models.CharField(max_length=30, blank=True)
    encaisseur = models.CharField(max_length=100, blank=True)

    def __str__(self):
        return f"{self.eleve} – {self.type_frais} : {self.montant} FCFA"
    class Meta:
        verbose_name = "Paiement"
        ordering = ['-date']

class IncidentDisciplinaire(models.Model):
    GRAVITES = [
        ('avertissement', 'Avertissement verbal'),
        ('blame', 'Blâme écrit'),
        ('exclusion_tmp', 'Exclusion temporaire'),
        ('exclusion_def', 'Exclusion définitive'),
    ]
    eleve = models.ForeignKey(Eleve, on_delete=models.CASCADE,
                             related_name='incidents')
    date = models.DateField()
    description = models.TextField()
    gravite = models.CharField(max_length=20, choices=GRAVITES)
    sanction = models.TextField(blank=True)
    notifie = models.BooleanField(default=False,
                                  verbose_name="Tuteur notifié")

    def __str__(self):
        return f"{self.eleve} – {self.get_gravite_display()} ({self.date})"
    class Meta: verbose_name = "Incident disciplinaire"

```

APRÈS ÉCRITURE : makemigrations + migrate

PHASE 3 – VUES CLÉS (ecole/views.py) (Jour 3-5)

VUE 1 – Tableau de bord direction

```

from django.db.models import Sum, Avg, Count, Q
from django.conf import settings

def dashboard(request):
    nb_eleves = Eleve.objects.filter(statut='inscrit').count()
    nb_classes = Classe.objects.count()
    nb_enseignants = Enseignant.objects.filter(aktif=True).count()

    # Frais scolaires : collecté vs attendu
    total_attendu = (Eleve.objects.filter(statut='inscrit').count()
                     * settings.FRAIS_SCOLARITE)
    total_collecte = (Paiement.objects
                     .filter(type_frais_nom__icontains='scolarité')
                     .aggregate(Sum('montant'))['montant__sum'] or 0)
    taux_recouvrement = round(total_collecte / total_attendu * 100, 1) if total_attendu else 0

    # Absences non justifiées ce mois
    from django.utils import timezone
    ce_mois = timezone.now().replace(day=1).date()
    absences_mois = AbsenceEleve.objects.filter(
        date_gte=ce_mois, justifiee=False).count()

    # Top 3 classes par moyenne générale
    top_classes = _moyennes_par_classe()[:3]

    return render(request, 'ecole/dashboard.html', {

```

```

        'nb_eleves':          nb_eleves,
        'nb_classes':        nb_classes,
        'nb_enseignants':    nb_enseignants,
        'total_attendu':      total_attendu,
        'total_collecte':     total_collecte,
        'taux_recouvrement':  taux_recouvrement,
        'absences_mois':      absences_mois,
        'top_classes':        top_classes,
        'annee_scolaire':     settings.ANNEE_SCOLAIRE,
    })

def _moyennes_par_classe():
    classes = Classe.objects.annotate(
        moyenne=Avg('eleve__notes__note')
    ).order_by('-moyenne')
    return [{'classe': str(c), 'moyenne': round(c.moyenne or 0, 2)}
            for c in classes]

VUE 2 – Bulletin de notes d'un élève


---


def bulletin_eleve(request, eleve_id, trimestre):
    eleve = get_object_or_404(Eleve, pk=eleve_id)
    matieres = Matiere.objects.all()
    lignes = []
    moyenne_generale = 0
    total_coeff = 0

    for m in matieres:
        notes_matiere = Note.objects.filter(
            eleve=eleve, matiere=m, trimestre=trimestre)
        if notes_matiere.exists():
            moy = notes_matiere.aggregate(Avg('note'))['note__avg']
            lignes.append({
                'matiere':      m,
                'notes':        notes_matiere,
                'moyenne':      round(moy, 2),
                'coefficient':  m.coefficient,
                'pts_ponderes': round(moy * float(m.coefficient), 2),
            })
            moyenne_generale += moy * float(m.coefficient)
            total_coeff      += float(m.coefficient)

    moy_gen = round(moyenne_generale / total_coeff, 2) if total_coeff else 0
    mention = _calculer_mention(moy_gen)
    rang = _calculer_rang(eleve, trimestre, moy_gen)

    return render(request, 'ecole/bulletin.html', {
        'eleve':      eleve,
        'trimestre':  dict(Note.TRIMESTRES)[trimestre],
        'lignes':     lignes,
        'moy_gen':    moy_gen,
        'mention':    mention,
        'rang':       rang,
    })

def _calculer_mention(moyenne):
    if moyenne >= 16: return 'Très Bien'
    if moyenne >= 14: return 'Bien'
    if moyenne >= 12: return 'Assez Bien'
    if moyenne >= 10: return 'Passable'
    return 'Insuffisant'

def _calculer_rang(eleve, trimestre, moy_gen):
    # Rang de l'élève dans sa classe pour ce trimestre
    camarades = (Note.objects
        .filter(eleve__classe=eleve.classe, trimestre=trimestre)
        .values('eleve')
        .annotate(moy=Avg('note'))
        .order_by('-moy'))
    ids_classes = [c['eleve'] for c in camarades]
    if eleve.pk in ids_classes:
        return ids_classes.index(eleve.pk) + 1
    return None

```

VUE 3 – Suivi des frais impayés

```
def frais_impayes(request):
    type_scol = TypeFrais.objects.filter(nom__icontains='scolarité').first()
    if not type_scol:
        messages.warning(request, "Type de frais 'Scolarité' non trouvé.")
        return redirect('dashboard')

    # Élèves qui ont payé
    eleves_ayant_paye = (Paielement.objects
                        .filter(type_frais=type_scol)
                        .values_list('eleve_id', flat=True).distinct())

    # Élèves inscrits qui n'ont PAS payé
    eleves_impayes = (Eleve.objects
                    .filter(statut='inscrit')
                    .exclude(pk__in=eleves_ayant_paye)
                    .select_related('classe', 'tuteur')
                    .order_by('classe', 'nom'))

    return render(request, 'ecole/frais_impayes.html', {
        'eleves': eleves_impayes,
        'nb_impayes': eleves_impayes.count(),
        'montant_manquant': eleves_impayes.count() * type_scol.montant_std,
        'type_frais': type_scol,
    })
```

PHASE 4 – TEMPLATES CLÉS (Jour 5-6)

templates/ecole/bulletin.html (extrait table de notes) :

```
<div class="bulletin-entete">
    <h2>BULLETIN DE NOTES – {{ trimestre }}</h2>
    <p><strong>Élève :</strong> {{ eleve.nom_complet }}</p>
    <p><strong>Classe :</strong> {{ eleve.classe }}</p>
    <p><strong>Année :</strong> {{ annee_scolaire }}</p>
</div>

<table class="table table-bordered">
    <thead class="table-dark">
        <tr>
            <th>Matière</th><th>Coeff.</th>
            <th>Notes obtenues</th><th>Moyenne /20</th><th>Pts pondérés</th>
        </tr>
    </thead>
    <tbody>
        {% for ligne in lignes %}
        <tr class="{% if ligne.moyenne < 10 %}table-warning{% endif %}">
            <td>{{ ligne.matiere.nom }}</td>
            <td>{{ ligne.coefficient }}</td>
            <td>{% for n in ligne.notes %}{{ n.note }}{% if not forloop.last %} / {% endif %}{% endfor %}</td>
            <td><strong>{{ ligne.moyenne }}</strong></td>
            <td>{{ ligne.pts_ponderes }}</td>
        </tr>
        {% endfor %}
    </tbody>
    <tfoot class="table-info">
        <tr>
            <td colspan="3"><strong>Moyenne Générale</strong></td>
            <td colspan="2">
                <strong>{{ moy_gen }}</strong> / 20 – {{ mention }}
                {% if rang %} (Rang : {{ rang }}e){% endif %}
            </td>
        </tr>
    </tfoot>
</table>

<button onclick="window.print()" class="btn btn-primary"> Imprimer</button>
```

templates/ecole/frais_impayes.html (extrait) :

```
<div class="alert alert-danger">
    <strong>{{ nb_impayes }} élèves</strong> n'ont pas réglé les frais de scolarité.
    Montant total manquant : <strong>{{ montant_manquant }} FCFA</strong>
</div>
```

```

</div>

<table class="table">
  <thead>
    <tr><th>Élève</th><th>Classe</th><th>Tuteur</th><th>Téléphone</th><th>Action</th></tr>
  </thead>
  <tbody>
    {% for e in eleves %}
    <tr>
      <td>{{ e.nom_complet }}</td>
      <td>{{ e.classe }}</td>
      <td>{{ e.tuteur|default:"-"}</td>
      <td>{{ e.tuteur.telephone|default:"-"}</td>
      <td>
        <a href="{% url 'saisir_paiement' e.pk %}" class="btn btn-sm btn-success">
          □ Enregistrer paiement
        </a>
      </td>
    </tr>
    {% endfor %}
  </tbody>
</table>

```

PHASE 5 – ANALYSE DATA SCIENCE ÉCOLE (Jour 6-7)

ANALYSE 1 – Distribution des moyennes par classe (histogramme)

```

import pandas as pd
import numpy as np
from django.db.models import Avg

def distribution_moyennes_classe(classe_id, trimestre='1'):
    notes_qs = (Note.objects
        .filter(eleve__classe_id=classe_id, trimestre=trimestre)
        .values('eleve_id', 'eleve_nom', 'eleve_prenom')
        .annotate(moyenne=Avg('note'))
        .order_by('-moyenne'))

    df = pd.DataFrame(list(notes_qs))
    if df.empty: return {}

    df['eleve'] = df['eleve_prenom'] + ' ' + df['eleve_nom']
    df['mention'] = df['moyenne'].apply(lambda m:
        'Très Bien' if m >= 16 else
        'Bien' if m >= 14 else
        'Assez Bien' if m >= 12 else
        'Passable' if m >= 10 else 'Insuffisant')

    stats = {
        'moyenne_classe': round(float(df['moyenne'].mean()), 2),
        'plus_haute': round(float(df['moyenne'].max()), 2),
        'plus_basse': round(float(df['moyenne'].min()), 2),
        'ecart_type': round(float(df['moyenne'].std()), 2),
        'taux_reussite_pct': round((df['moyenne'] >= 10).sum() / len(df) * 100, 1),
        'repartition_mentions': df['mention'].value_counts().to_dict(),
        'classement': df[['eleve', 'moyenne', 'mention']].to_dict('records'),
    }
    return stats

```

ANALYSE 2 – Corrélation assiduité / résultats scolaires

```

def correlation_assiduite_resultats(trimestre='1'):
    """
    Vérifie si les élèves les plus absents ont de moins bons résultats.
    Résultat data science : coefficient de corrélation de Pearson.
    """
    eleves = Eleve.objects.filter(statut='inscrit')
    donnees = []

    for e in eleves:
        nb_absences = AbsenceEleve.objects.filter(
            eleve=e, justifiee=False).count()
        moy = Note.objects.filter(

```

```

        eleve=e, trimestre=trimestre
    ).aggregate(Avg('note'))['note__avg']

    if moy is not None:
        donnees.append({
            'eleve':      e.nom_complet(),
            'absences':    nb_absences,
            'moyenne':      float(moy),
        })

df = pd.DataFrame(donnees)
if len(df) < 5: return None

correlation = df['absences'].corr(df['moyenne'])
interpretation = (
    "forte corrélation négative (les absences nuisent clairement)"
    if correlation < -0.6 else
    "corrélation modérée" if correlation < -0.3 else
    "faible corrélation"
)
return {
    'coefficient_pearson': round(correlation, 3),
    'interpretation':      interpretation,
    'nb_eleves_analyses':  len(df),
    'donnees':             df.sort_values('absences', ascending=False)
                           .head(10).to_dict('records'),
}

```

ANALYSE 3 – Préviation taux de réussite au BEPC / BAC

```

from sklearn.linear_model import LogisticRegression
import numpy as np

def predire_reussite_examen(classe_id):
    """
    Prédit la probabilité de réussite au BEPC/BAC
    basée sur la moyenne du 1er et 2ème trimestre.
    Nécessite au moins 2 trimestres de données.
    """
    eleves = Eleve.objects.filter(classe_id=classe_id, statut='inscrit')
    X, y = [], []

    for e in eleves:
        moy_t1 = Note.objects.filter(eleve=e, trimestre='1').aggregate(Avg('note'))['note__avg']
        moy_t2 = Note.objects.filter(eleve=e, trimestre='2').aggregate(Avg('note'))['note__avg']
        if moy_t1 and moy_t2:
            X.append([float(moy_t1), float(moy_t2)])
            y.append(1 if (moy_t1 + moy_t2) / 2 >= 10 else 0)

    if len(X) < 10:
        return {'erreur': 'Pas assez de données (minimum 10 élèves avec 2 trimestres)'}

    modele = LogisticRegression()
    modele.fit(X, y)

    # Prédire pour chaque élève
    probas = modele.predict_proba(X)
    resultats = []
    for i, e in enumerate([el for el in eleves if i < len(X)]):
        resultats.append({
            'eleve':      e.nom_complet(),
            'prob_reussite': round(probas[i][1] * 100, 1),
            'prediction':    'Admis probable' if probas[i][1] > 0.5 else 'À risque',
        })

    taux_global = round(sum(y) / len(y) * 100, 1)
    return {
        'taux_reussite_actuel': taux_global,
        'predictions':         sorted(resultats, key=lambda x: x['prob_reussite'], reverse=True),
        'nb_eleves':            len(X),
    }

```


FIXTURE – Classes type collège (fixtures/classes_college.json) :

```
[
  {"model":"ecole.classe","pk":1,"fields":{"niveau":"6e","lettre":"A","effectif_max":50,"salle":"S01"}},
  {"model":"ecole.classe","pk":2,"fields":{"niveau":"6e","lettre":"B","effectif_max":50,"salle":"S02"}},
  {"model":"ecole.classe","pk":3,"fields":{"niveau":"5e","lettre":"A","effectif_max":48,"salle":"S03"}},
  {"model":"ecole.classe","pk":4,"fields":{"niveau":"4e","lettre":"A","effectif_max":45,"salle":"S04"}},
  {"model":"ecole.classe","pk":5,"fields":{"niveau":"3e","lettre":"A","effectif_max":45,"salle":"S05"}}
]
```

FIXTURE – Types de frais (fixtures/types_frais.json) :

```
[
  {"model":"ecole.typefrais","pk":1,"fields":{"nom":"Frais de scolarité","montant_std":45000,"obligatoire":true}},
  {"model":"ecole.typefrais","pk":2,"fields":{"nom":"Tenue scolaire","montant_std":15000,"obligatoire":true}},
  {"model":"ecole.typefrais","pk":3,"fields":{"nom":"Manuels scolaires","montant_std":12000,"obligatoire":false}},
  {"model":"ecole.typefrais","pk":4,"fields":{"nom":"Cotisation APEE","montant_std":5000,"obligatoire":false}}
]
```

FIXTURE – Matières collège (fixtures/matieres_college.json) :

```
[
  {"model":"ecole.matiere","pk":1,"fields":{"nom":"Mathématiques","code":"MATH","coefficient":4.0}},
  {"model":"ecole.matiere","pk":2,"fields":{"nom":"Français","code":"FRAN","coefficient":4.0}},
  {"model":"ecole.matiere","pk":3,"fields":{"nom":"Sciences de la Vie et de la Terre","code":"SVT","coefficient":2.0}},
  {"model":"ecole.matiere","pk":4,"fields":{"nom":"Histoire-Géographie","code":"HIST","coefficient":2.0}},
  {"model":"ecole.matiere","pk":5,"fields":{"nom":"Physique-Chimie","code":"PC","coefficient":2.0}},
  {"model":"ecole.matiere","pk":6,"fields":{"nom":"Anglais","code":"ANG","coefficient":2.0}},
  {"model":"ecole.matiere","pk":7,"fields":{"nom":"Éducation Physique","code":"EPS","coefficient":1.0}},
  {"model":"ecole.matiere","pk":8,"fields":{"nom":"Arts Plastiques","code":"ART","coefficient":1.0}}
]
```

CHARGER TOUTES LES FIXTURES :

```
python manage.py loaddata fixtures/classes_college.json
python manage.py loaddata fixtures/types_frais.json
python manage.py loaddata fixtures/matieres_college.json
```

CHECKLIST DE LIVRAISON – ÉCOLE

- ☐ NOM_ECOLE, DEVISE_ECOLE, ANNEE_SCOLAIRE renseignés dans settings.py
- ☐ Logo de l'école remplacé (même nom de fichier)
- ☐ Classes, matières et types de frais chargés via fixtures
- ☐ Compte superutilisateur créé pour le Directeur
- ☐ Comptes séparés : Directeur (tout), Secrétaire (saisie), Enseignant (notes)
- ☐ Bulletin d'un élève générable et imprimable
- ☐ Liste des frais impayés visible avec bouton d'action
- ☐ Corrélation assiduité/résultats calculable depuis le dashboard
- ☐ Export Excel des résultats par classe fonctionnel
- ☐ db.sqlite3 sauvegardé sur clé USB avant remise
- ☐ Formation 3h : inscription élève, saisie note, bulletin, frais impayés

DURÉE TOTALE ESTIMÉE : 8 jours ouvrés pour un développeur Niveau 1

Développé par GCI – Gykhamine Concept Investigation

Brazzaville, République du Congo

"Une capsule adaptée vaut mieux qu'un logiciel importé inadapté."
